# Sixth Implementation Iteration: Space Allocation

## with Day-Based Time Slots

Software Design / Iteration Report

Name: Najib Attar
Date: March 2026

## 1. Iteration Goal

The goal of this sixth implementation iteration is to improve the time representation used in the prototype by extending the `TimeSlot` model with day information. In previous iterations, time slots were represented only by start and end hours, which meant that requests with the same hours on different days were incorrectly treated as conflicting.

In this iteration, each time slot is extended with a day field. The existing decision rules remain unchanged. A request is still approved only if the requested space satisfies the capacity requirement and the requested time slot or all requested occurrences satisfy the availability rule. However, the availability rule now checks both day and time when detecting overlaps.

## 2. Changes from Iteration 5

Compared with the fifth iteration, the following changes were introduced:

- Extended the `TimeSlot` class by adding a day attribute
- Updated the overlap checking logic so that two time slots conflict only if they occur on the same day and their hours overlap
- Updated one-time and recurring requests so that all requested occurrences include day information
- Preserved the same request types:
  - `OneTimeRequest`
  - `RecurringRequest`
- Preserved the same supported space types:

  - `Classroom`
  - `Laboratory`
  - `MeetingRoom`
- Preserved the same rules:
  - `AvailabilityRule`
  - `CapacityRule`

## 3. Scenario

A user submits either a one-time request or a recurring request for a specific space resource. A one-time request contains a single time slot, while a recurring request contains multiple time slots representing repeated occurrences of the same booking.

In this iteration, each time slot includes both the day and the time interval. The system checks whether the requested space is available during the requested day and time slot or time slots and whether the space capacity is sufficient for the participant count.

A one-time request is approved if its single occurrence passes all rules. A recurring request is approved only if all occurrences pass all rules. Otherwise, the request is rejected.

## 4. Modules and Their Tasks

### A. Request Management Module

- Receive one-time or recurring space requests from the user
- Create and store `OneTimeRequest` or `RecurringRequest`
- Include request details:
  - requester
  - requested space
  - requested time slot or time slots with day information
  - participant count
- Send the request to the Rule Evaluation Module

### B. Resource Management Module

- Manage all supported space types

- Provide information for `Classroom`, `Laboratory`, and `MeetingRoom`
- Provide identity, type, name, and capacity of the requested space

### C. Rule Evaluation Module

- Apply `AvailabilityRule`
- Apply `CapacityRule`
- For recurring requests, evaluate all requested occurrences
- Detect conflicts by checking both day and hour overlap
- Return the final decision: Approved or Rejected

### D. Allocation Management Module

- Create one allocation for a one-time request if all rules pass
- Create multiple allocations for a recurring request if all occurrences pass
- Update request status to Approved or Rejected

## 5. Iteration Comparison

| **Iteration** | **Request Types** | **Resource Types** | **Rules Implemented** |
|---|---|---|---|
| Iteration 1 | OneTimeRequest | Classroom | AvailabilityRule |
| Iteration 2 | OneTimeRequest | Classroom | AvailabilityRule + CapacityRule |
| Iteration 3 | OneTimeRequest | Classroom + Laboratory | AvailabilityRule + CapacityRule |
| Iteration 4 | OneTimeRequest | Classroom + Laboratory + MeetingRoom | AvailabilityRule + CapacityRule |
| Iteration 5 | OneTimeRequest + RecurringRequest | Classroom + Laboratory + MeetingRoom | AvailabilityRule + CapacityRule |
| Iteration 6 | OneTimeRequest + RecurringRequest | Classroom + Laboratory + MeetingRoom | AvailabilityRule + CapacityRule |

## 6. Updated Class Diagram

Figure 1 shows the updated simplified class diagram for the sixth implementation iteration.

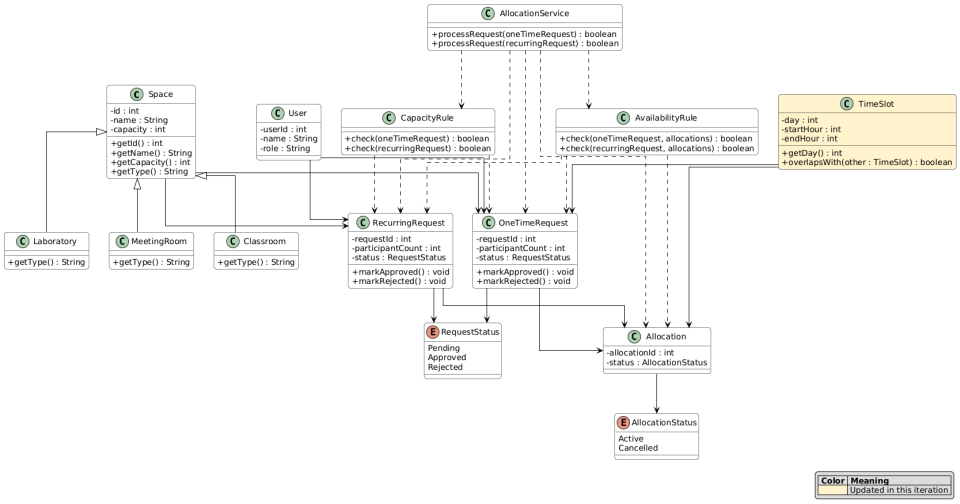


Figure 1: Updated Simplified Class Diagram

## 7. Updated Workflow Diagram

Figure 2 shows the workflow for the sixth iteration of the space allocation process.

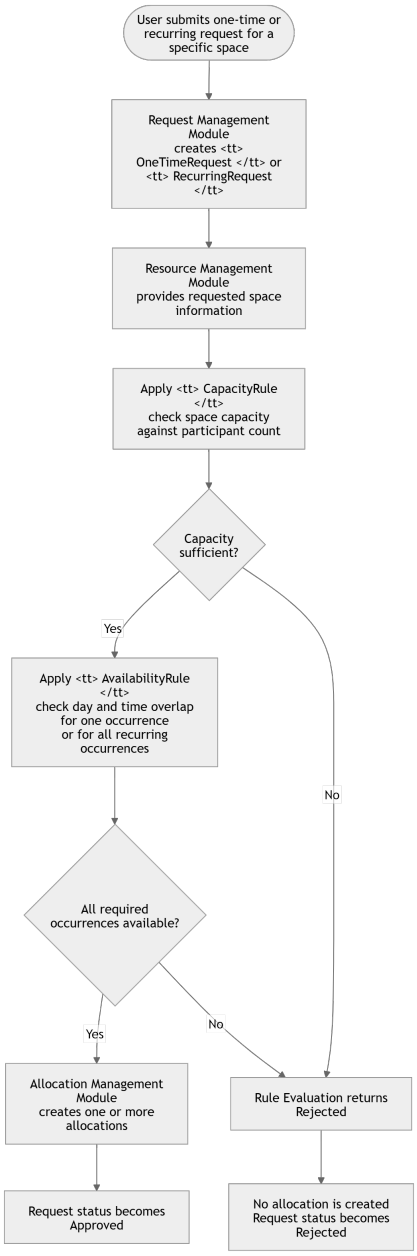


Figure 2: Updated Workflow Diagram

## 8. Prototype Implementation

To validate the design of this iteration, the C++ prototype was extended by modifying the `TimeSlot` class to include a day attribute. The overlap checking logic was updated so that two time slots are considered conflicting only if they occur on the same day and their hours overlap.

The allocation service continues to support both one-time and recurring requests. For recurring requests, the service checks the capacity rule once and the availability rule across all requested occurrences using the updated day-based overlap logic.

If all occurrences pass, multiple allocations are created, one for each occurrence. If one or more occurrences fail, the recurring request is rejected.

## 9. Summary

This sixth implementation iteration improves the prototype by extending the time model with day-based time slots while preserving the existing request types, space types, and decision rules. The system can now correctly distinguish requests that occur at the same hours on different days.

This iteration demonstrates controlled iterative development by refining the time representation while preserving the same allocation architecture. Future iterations can continue this approach by adding more advanced scheduling constraints or calendar-related rules.